

RISC-V Single - Cycle Microarchitecture

A PROJECT REPORT

Submitted by:

Meet Rudani – 20BEEEC30006

Parth vaghasiya – 20BEEEC30007

In fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

Electronics and Communication



LDRP Institute of Technology and Research, Gandhinagar

Kadi Sarva Vishwavidyalaya, Gandhinagar

***LDRP INSTITUTE OF TECHNOLOGY AND RESEARCH,
GANDHINAGAR***

Electronics and Communication Engineering Department



CERTIFICATE

This is to certify that the Project Work entitled “**RISC-V Single Cycle Microarchitecture**” has been carried out by **Meet Rudani (20BEEC30006)**, **Parth Vaghasiya (20BEEC30007)** under my guidance in fulfilment of the degree of Bachelor of Engineering in **Electronics and Communication** (7th Semester) of Kadi Sarva Vishwavidyalaya University, Gandhinagar during the academic year 2023.

Dr. Kruti P. Thakore

Name & Sign

Internal Guide

LDRP-ITR

Dr. Shridhar Mendhe

Name & Sign

Head of the department

LDRP-ITR

ACKNOWLEDGEMENT

The project report on "**RISC-V Single Cycle Microarchitecture**" was taken up as 7th Semester Project. Here I wish to express my sincere thankfulness to those people who have helped me throughout my project work. I would like to thank the people who stood by me to prepare this-project report. I sincerely feel that the credit of the project work cannot be narrowed down to only one individual.

This project work is an integrated effort of all those person concerned; through co-operative and effective guidance I would achieve its completion. I would like to thank **Dr. Kruti P. Thakore**, Prof. EC Department, for making available all facilities in fulfilling the requirement for project work. I also take this opportunity to express a deep sense of gratitude to **Dr. SHRIDHAR MENDHE**, HOD EC Department for his cordial support, valuable information and guidance, which helped us in completing this task through various stages.

I am deeply appreciative of our parents and our colleagues for their constant support, encouragement and bearing with us during the period of our project work. Finally, it is our foremost duty to thank all our respondents, who helped us to complete our Project, without them this project was not possible.

Table of Contents:

1.1 Abstract

CHAPTER – 1: Introduction

1.1 Introduction

1.2 Merits

1.3 Block diagram

CHAPTER – 2: History and some Challenges

1.1 History

1.2 Background or problem

CHAPTER –3: Theory

1.1 Arithmetic Logic Unit (ALU)

1.1.1 Introduction

1.1.2 Block diagram

2.1 Control Unit

2.1.1 Introduction

2.1.1.1 R-Type Instructions

2.1.1.2 I-Type Instructions

2.1.1.3 S-Type Instructions

2.1.1.4 B-Type Instructions

2.1.2 Block Diagram

2.1.2.1 MAIN Decoder

2.1.2.2 ALU Decoder

3.1 Designing Microarchitecture-I

3.1.1 State Elements

4.1 Designing Microarchitecture-II

4.1.1 Introduction

4.1.2 DataPath for Load Word Instruction

5.1 Designing Microarchitecture-III

5.1.1 Introduction

5.1.2 DataPath for Store Word Instruction

6.1 Designing Microarchitecture-IV

6.1.1 Introduction

6.1.2 DataPath for R-Type Instruction

CHAPTER – 4: Programming

1.1 ALU.v

2.1 ALU_Decoder.v

3.1 Control_Unit_Top.v

4.1 Data_Memory.v

5.1 Instruction_Memory.v

6.1 Main_Decoder.v

7.1 Mux.v

8.1 PC.v

9.1 PC_Adder.v

10.1 Register_File.v

11.1 Sign_Extend.v

12.1 Single_Cycle_Top.v

13.1 Single_Cycle_Top_Tb.v

CHAPTER – 5: Further Modifications

CHAPTER – 6: Applications

CHAPTER – 7: Conclusion

1.1 Conclusion

References.

Abstract

This report presents the design and implementation of a single-cycle RISC-V microarchitecture. RISC-V is an open-source instruction set architecture (ISA) that has gained popularity due to its simplicity and flexibility. The microarchitecture is designed to execute instructions in a single clock cycle, making it suitable for applications that require low-latency processing.

The microarchitecture includes components such as an instruction memory, a data memory, a register file, and an arithmetic logic unit (ALU). Instructions are fetched from the instruction memory, decoded, and executed in a single clock cycle. The microarchitecture supports the base integer instruction set (RV32I) of the RISC-V ISA.

The report discusses the design decisions and challenges faced during the implementation of the microarchitecture. It also presents the Verilog code for the microarchitecture and provides simulation results to demonstrate its functionality. Overall, the single-cycle RISC-V microarchitecture provides a basic framework for understanding the fundamental concepts of RISC-V processor design.

CHAPTER -1

INTRODUCTION

1.1 INTRODUCTION

The RISC-V instruction set architecture (ISA) has emerged as a popular choice for designing efficient and scalable processors. One of the key features of the RISC-V ISA is its modular design, which allows for different implementations based on specific application requirements. In this report, we present the design and implementation of a single-cycle RISC-V microarchitecture, focusing on the RV32I base integer instruction set.

The single-cycle microarchitecture is designed to execute each instruction in a single clock cycle, making it suitable for applications that require low-latency processing. The design includes components such as an instruction memory, a data memory, a register file, and an arithmetic logic unit (ALU). Instructions are fetched from the instruction memory, decoded, and executed in a single clock cycle.

The primary goal of this project is to provide a basic framework for understanding the key concepts of RISC-V processor design, including instruction fetching, decoding, and execution. By implementing a single-cycle microarchitecture, we aim to demonstrate the simplicity and efficiency of the RISC-V ISA for basic integer operations.

In this report, we will discuss the design decisions and challenges faced during the implementation of the single-cycle microarchitecture. We will also present the Verilog code for the microarchitecture and provide simulation results to validate its functionality. Overall, this project serves as a foundational study in RISC-V processor design and provides a starting point for more complex implementations based on the RISC-V ISA.

2.1 Merits:

Performance Evaluation: Porting allows for the evaluation of the microarchitecture's performance on different platforms, enabling designers to assess its efficiency and effectiveness in different environments.

Platform Compatibility: Porting ensures compatibility with new platforms, enabling the microarchitecture to be used in a wider range of applications and environments.

Technology Exploration: Porting can facilitate exploration into new technologies and methodologies, providing insights into their applicability and potential advantages.

Enhanced Flexibility: Porting increases the flexibility of the microarchitecture, allowing it to adapt to new requirements and specifications.

Improved Scalability: Porting enables the microarchitecture to be scaled to different sizes and complexities, making it suitable for a variety of applications and use cases.

Cost Savings: Porting can lead to cost savings by reusing existing designs and infrastructure, reducing the need for new development from scratch.

Educational Value: Porting can be a valuable educational exercise, allowing students and researchers to gain hands-on experience with microarchitecture design and implementation.

Overall, porting a RISC-V Single Cycle Microarchitecture offers numerous benefits, including improved performance, compatibility, flexibility, scalability, cost savings, and educational value.

CHAPTER -2

History and some Challenges

1.1 History

The history of porting the RISC-V Single Cycle Microarchitecture involves the evolution of the RISC-V ISA, the development of the Single Cycle Microarchitecture concept, and its implementation in various projects and educational endeavors.

The RISC-V ISA was first introduced in 2010 at the University of California, Berkeley, as an open-source instruction set architecture designed to be simple, modular, and extensible. The ISA was developed to address the shortcomings of existing ISAs and to provide a platform for innovation in computer architecture. The RISC-V ISA quickly gained popularity due to its openness and flexibility, leading to its adoption in a wide range of applications.

One of the key concepts in the RISC-V ISA is the Single Cycle Microarchitecture, which aims to execute each instruction in a single clock cycle. This approach simplifies the design of the processor and provides a deterministic execution model, making it ideal for educational purposes. The Single Cycle Microarchitecture concept was inspired by the early RISC processors, such as the MIPS R2000, which also aimed to execute instructions in a single clock cycle.

The development of the RISC-V Single Cycle Microarchitecture involved several key steps, including the design of the instruction set, the implementation of the microarchitecture, and the validation of the design through simulation and testing. The microarchitecture includes components such as the instruction fetch unit, instruction decode unit, execution unit, memory unit, and write-back unit, all of which work together to execute instructions in a single clock cycle.

Several projects and educational initiatives have implemented the RISC-V Single Cycle Microarchitecture, including the RISC-V Hardware Development Guide and the RISC-V Single Cycle CPU project on GitHub. These projects provide resources and documentation for implementing the Single Cycle Microarchitecture and demonstrate its effectiveness as a teaching tool for computer architecture.

Overall, the history of porting the RISC-V Single Cycle Microarchitecture reflects the evolution of the RISC-V ISA and its adoption in various educational and research projects. The Single Cycle Microarchitecture concept continues to be a

valuable tool for teaching computer architecture and exploring the principles of the RISC-V ISA.

2.1 Background and Problem

The RISC-V (Reduced Instruction Set Computing - V) architecture has emerged as a promising open-source instruction set architecture (ISA) due to its simplicity, flexibility, and scalability. It was developed at the University of California, Berkeley, and is now maintained by the RISC-V Foundation, a non-profit organization. One of the key features of RISC-V is its modular design, which allows for different implementations ranging from simple microcontrollers to complex multi-core processors. This flexibility has led to its adoption in a wide range of applications, from embedded systems to high-performance computing.

The Single Cycle Microarchitecture is a fundamental implementation approach for executing instructions in a RISC-V processor. In this approach, each instruction is executed in a single clock cycle, providing a simple and deterministic execution model. This simplicity makes it an attractive choice for educational purposes, as it allows students to understand the basic principles of computer architecture without the complexity of pipelining or out-of-order execution.

The development of a RISC-V Single Cycle Microarchitecture involves several key components, including the instruction fetch unit, instruction decode unit, execution unit, memory unit, and write-back unit. These components work together to fetch, decode, execute, and write back the results of instructions in a single clock cycle. While the Single Cycle Microarchitecture is not as efficient as more complex implementations, such as pipelined or superscalar architectures, it provides a solid foundation for understanding the principles of computer architecture.

In recent years, there has been a growing interest in RISC-V due to its open nature and the availability of open-source tools and implementations. This has led to the development of a wide range of RISC-V processors, from simple microcontrollers to high-performance processors. The Single Cycle Microarchitecture plays an important role in this ecosystem by providing a simple and accessible implementation approach that can be used for education, research, and prototyping.

Overall, the RISC-V Single Cycle Microarchitecture is a key building block in the RISC-V ecosystem, providing a simple and efficient implementation approach for executing instructions in a RISC-V processor. Its simplicity and accessibility make

it an attractive choice for educators, students, and researchers looking to explore the principles of computer architecture and the RISC-V ISA.

Porting a RISC-V Single Cycle Microarchitecture to a new platform or technology can pose several challenges, ranging from architectural compatibility to performance optimization.

Some of the key challenges include:

Instruction Set Compatibility: One of the primary challenges in porting a RISC-V Single Cycle Microarchitecture is ensuring compatibility with the RISC-V instruction set architecture (ISA). The new platform may have different memory models, addressing modes, or instruction formats, requiring modifications to the microarchitecture to support these differences while maintaining compatibility with the RISC-V ISA.

Clock Speed and Timing: Achieving the desired clock speed and timing characteristics can be challenging, especially when porting to a new technology or platform with different timing constraints. The Single Cycle Microarchitecture relies on strict timing requirements for each instruction, and ensuring these requirements are met on the new platform can be non-trivial.

Resource Constraints: Porting a Single Cycle Microarchitecture to a new platform may require adapting to different resource constraints, such as limited hardware resources or different memory hierarchies. This may require redesigning certain components of the microarchitecture to be more resource-efficient or to make better use of the available resources.

Memory System Design: The memory system design plays a critical role in the performance of the microarchitecture. Porting to a new platform may require redesigning the memory system to optimize for the new platform's memory hierarchy, bandwidth, and latency characteristics.

Power Efficiency: Power efficiency is becoming increasingly important in modern computing systems. Porting a Single Cycle Microarchitecture to a new platform may require optimizing for power efficiency, which may involve redesigning certain components of the microarchitecture or implementing power-saving techniques.

Testing and Verification: Ensuring the correctness and reliability of the ported microarchitecture is crucial. Testing and verifying the functionality of the microarchitecture on the new platform can be challenging and may require the development of new test cases and verification methodologies.

Performance Optimization: Finally, optimizing the performance of the ported microarchitecture on the new platform is a key challenge. This may involve identifying and addressing performance bottlenecks, tuning the microarchitecture for the new platform's characteristics, and potentially redesigning certain components for better performance.

In conclusion, porting a RISC-V Single Cycle Microarchitecture to a new platform or technology poses several challenges that require careful consideration and expertise in computer architecture and microarchitecture design. Addressing these challenges effectively is essential to ensure the successful porting of the microarchitecture and the optimal performance of the resulting system.

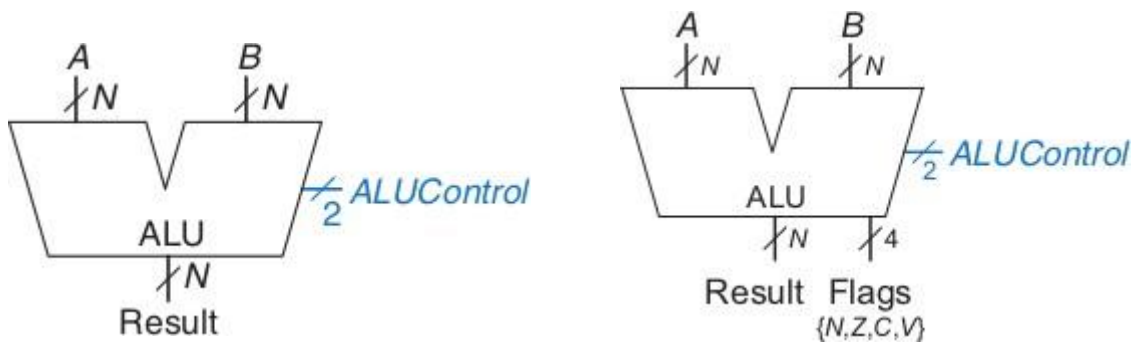
CHAPTER -3

Theory

1.1 Arithmetic Logic Unit (ALU)

1.1.1 Introduction

We will design an ALU that can perform a subset of the ALU operations of a full Processor ALU.



In this exercise, we will develop an ALU that will take two 2-inputs, A and B and is able to execute the following instructions:

ALUControl	Instruction
000 (add)	lw, sw
001 (subtract)	beq
000 (add)	add
001 (subtract)	sub
101 (set less than)	slt
011 (or)	or
010 (and)	and

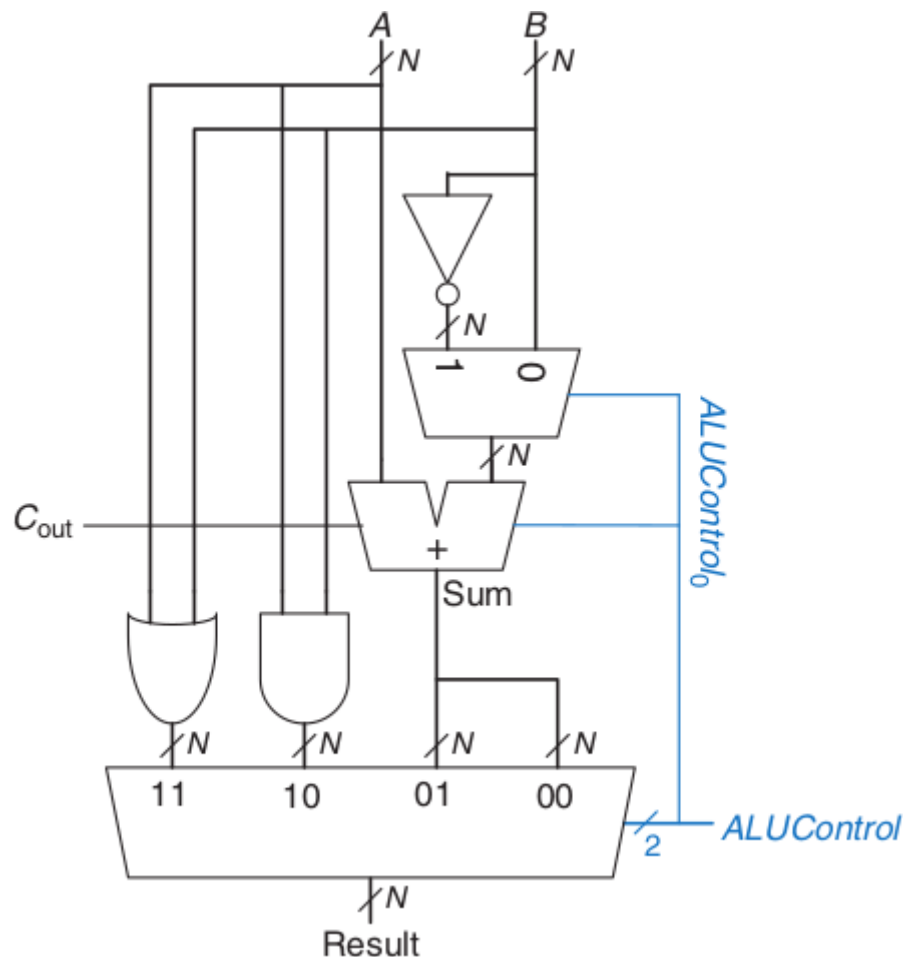
The ALU will generate a 32-bit output that we will call ‘Result’ and an additional 1-bit flag ‘Zero’ that will be set to ‘logic-1’ if all the bits of ‘Result’ are 0. The different operations will be selected by a 3-bit control signal called ‘ALUControl’ according to the following table.

For example, when the ‘ALUControl’ input is ‘011’, the function $\text{Result} = A \text{ or } B$ should be calculated. It is easy to see that there are many values of ‘ALUControl’ for which no operation has been defined. It is not very important what the circuit does when ‘ALUControl’ has these values, since the ‘Result’ will simply be ignored in these cases. You can use this to your advantage to simplify the circuit. Right now, the described operations may look random, but once we learn more about the Instruction set architecture, these choices will make more sense.

1.1.2 Block diagram

The first order of business should be drawing a block diagram. The following is one approach to analyzing what is needed and come up with a block diagram. You are free to follow this example or come up with your own ideas. It is just important that you think about how the circuit should be implemented. Let us first examine the different commands. You should see that we have two types of instructions. The three instructions **add**, **sub**, and **slt** are arithmetic operations, whereas the two remaining **and**, **or** are logical operations. Therefore, we have two separate groups of operations. Now let us look at the figure above and determine for which values of ALUControl we perform an operation from which

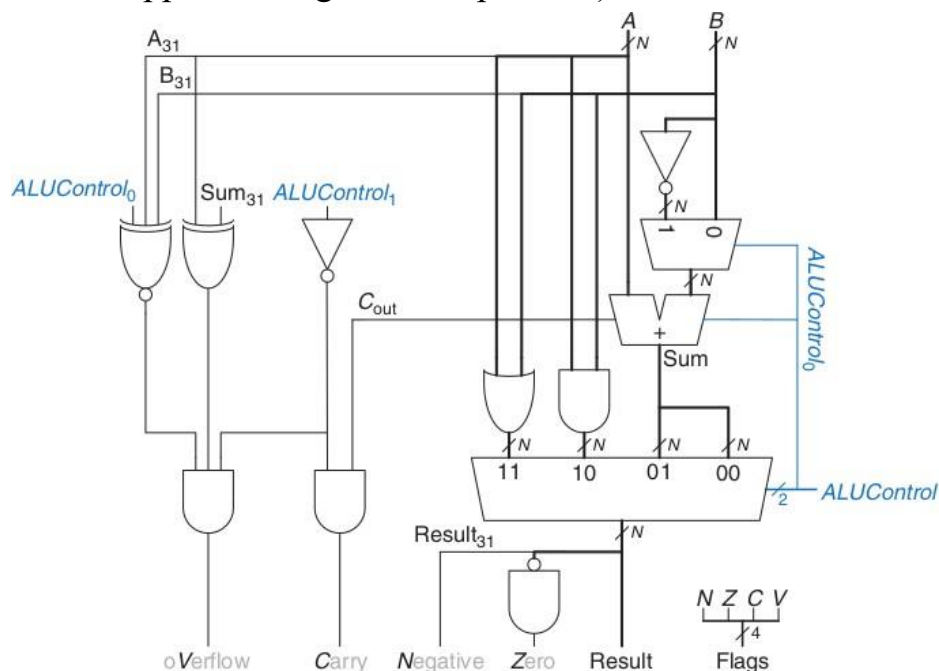
group. It should be pretty clear that when ALUControl[1] is logic-1 we select a logic operation and when ALUControl[1] is logic-0 we have an arithmetic operation. This means that the output of either group can be selected by a 4-input multiplexer that is controlled by ALUControl[1:0]. Figure below shows this distribution.



Now we can take a look at the two groups individually. In the first group we realize that we have an addition (add) or a subtraction (sub, slt). We again could observe that ALUControl[0] is logic-0 for additions and logic-1 for subtractions.. Figure below shows this arrangement.

Finally, we also have to add a small circuit that will generate the zero output when the result equals all zeroes. This method of breaking a larger block successively into smaller pieces is called Divide and Conquer and is one of the most important tricks that allow hardware engineers to design very complex circuits.

Note that in the above example, there are many values of ALUControl where the circuit would perform ‘strange’ operations (100 for example). This is not important because the circuit specification that was given to us said that these inputs were not relevant (this is probably because it can be guaranteed that these inputs will not appear during normal operation).



Comparison	Signed	Unsigned
=	Z	Z
≠	$\bar{Z}$	$\bar{Z}$
<	$N \oplus V$	$\bar{C}$
≤	$Z + (N \oplus V)$	$Z + \bar{C}$
>	$\bar{Z} \cdot (\bar{N} \oplus \bar{V})$	$\bar{Z} \cdot C$
≥	$\overline{(N \oplus V)}$	C

2.1 Control Unit

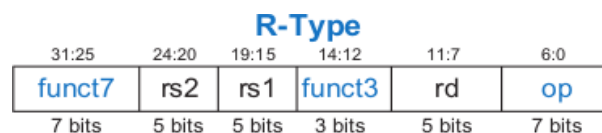
2.1.1 Introduction

RISC-V uses 32-bit instructions. RISC-V consists of defining the following instruction formats: R-type, I-type, S-Type, B-Type, U-type, and J-type. R-type instructions operate on three registers. I-type, S-type and B-type instructions operate on two registers and a 12-bit immediate. U-type and J-type (jump) instructions operate on one 20-bit immediate.

2.1.1.1 R-Type Instructions

The name R-type is short for register-type. R-type instructions use three registers as operands: two as sources, and one as a destination. The figure below shows the R-type machine instruction format. The 32-bit instruction has six fields: **op**, **rs1**, **rs2**, **rd**, **funct3**, and **funct7**. Each field is five or seven bits, as indicated. The operation the instruction performs is encoded in the three fields highlighted in blue: **op** (also called opcode or operation code) and **funct3** and **funct7** (also called the function). All R-type instructions have an opcode of **33**.

The specific R-type operation is determined by the function fields. The operands are encoded in the three fields: **rs1**, **rs2**, and **rd**. The first two registers, **rs1**, and **rs2** are the source registers; **rd** is the destination register.

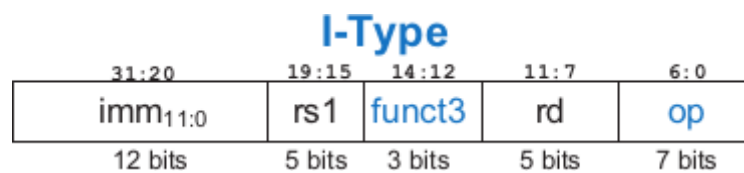


0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

Instruction Format for some of the R-type instructions is shown below:

2.1.1.2 I-Type Instructions

The name I-type is short for immediate-type. I-type instructions use two register operands and one immediate operand. The figure below shows the I-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rd, funct3, and imm. The first three fields, op, rs1, and rd, are like those of R-type instructions. The imm field holds the 12-bit immediate. The funct3 holds the operation to be performed. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the two fields rs1, and imm. rs1 and imm are always used as source operands. rd is used as a destination.



imm _{11:0}		rs1	000	rd	0000011	LB
imm _{11:0}		rs1	001	rd	0000011	LH
imm _{11:0}		rs1	010	rd	0000011	LW
imm _{11:0}		rs1	100	rd	0000011	LBU
imm _{11:0}		rs1	101	rd	0000011	LHU
imm _{11:0}		rs1	000	rd	0010011	ADDI
imm _{11:0}		rs1	010	rd	0010011	SLTI
imm _{11:0}		rs1	011	rd	0010011	SLTIU
imm _{11:0}		rs1	100	rd	0010011	XORI
imm _{11:0}		rs1	110	rd	0010011	ORI
imm _{11:0}		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI

Instruction format for some of the I-type instructions are shown below:

2.1.1.3 S-Type Instructions

The name S-type is short for store-type. S-type instructions use two register operands and one immediate operand. Figure below shows the S-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rs2, funct3 and imm. The first three fields op, rs1, and rs2 are like those of R-type instructions. The imm fields hold the 12-bit immediate. The 12-bit immediate is split into two sets as shown below. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the three fields rs1, rs2 and imm.



imm	11:5		rs2	rs1	000	imm	4:0	0100011	SB
imm	11:5		rs2	rs1	001	imm	4:0	0100011	SH
imm	11:5		rs2	rs1	010	imm	4:0	0100011	SW

Instruction format for some of the S-type instructions are shown below:

2.1.1.4 B-Type Instructions

The name B-type is short for branch-type. This format is used only with branch instructions. B-type instructions use two register operands and one immediate operand. Figure below shows the B-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rs2, funct3 and imm. The first three fields op, rs1, and rs2 are like those of R-type instructions. The imm fields hold the 12-bit immediate. The 12-bit immediate is split into two sets as shown below. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the three fields rs1, rs2 and imm.



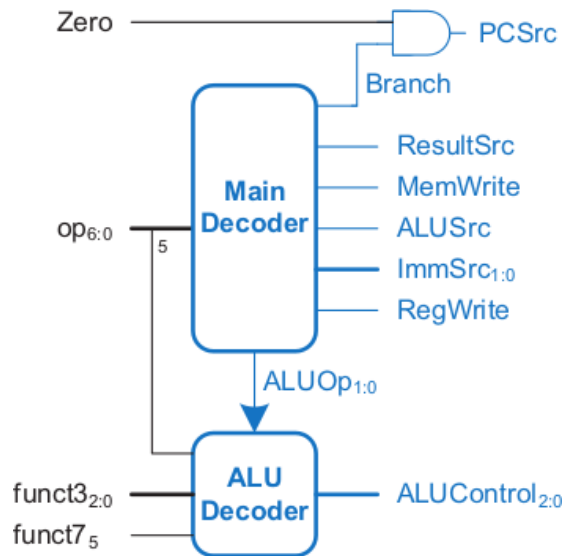
imm	12 10:5		rs2	rs1	000	imm	4:1 11	1100011	BEQ
imm	12 10:5		rs2	rs1	001	imm	4:1 11	1100011	BNE
imm	12 10:5		rs2	rs1	100	imm	4:1 11	1100011	BLT
imm	12 10:5		rs2	rs1	101	imm	4:1 11	1100011	BGE
imm	12 10:5		rs2	rs1	110	imm	4:1 11	1100011	BLTU
imm	12 10:5		rs2	rs1	111	imm	4:1 11	1100011	BGEU

Instruction format for some of the B-type instructions are shown below:

To interpret machine language, one must decipher the fields of each 32-bit instruction word. Different instructions use different formats, but all formats start with a 7-bit opcode field. Thus, the best place to begin is to look at the opcode.

2.1.2 Block Diagram

The control unit computes the control signals based on the opcode and funct fields of the instruction, Instr[31:25], Instr[14:12] and Instr[6:0]. Most of the control information comes from the opcode, but for further operations function fields are used. Thus, we will simplify our design by factoring the control unit into two blocks of combinational logic, as shown in Figure below



2.1.2.1 MAIN Decoder

The table below is a truth table for the main decoder that summarizes the control signals as a function of the opcode. All R-type instructions use the same main decoder values; they differ only in the ALU decoder output. Recall that, for instructions that do not write to the register file (e.g., S-type and B-type), the ResultSrc control signals is don't care (X); the address and data to the register write port do not matter because RegWrite is not asserted. The logic for the decoder can be designed using your favorite techniques for combinational logic design.

Instruction	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	x	0	00
R-type	0110011	1	xx	0	0	0	0	10
beq	1100011	0	10	0	0	x	1	01

ALU Decoder

The main decoder computes most of the outputs from the opcode. It also determines a 2-bit ALUOp signal. The ALU decoder uses this ALUOp signal in conjunction with the funct field and opcode bit to compute ALUControl. The meaning of the ALUOp signal is given in Table below

ALUOp	Meaning
00	add
01	subtract
10	look at funct fields and opcode bit
11	N/A

Table below is a truth table for the ALU decoder. The logic of ALUControl was

covered in the above chapter. When ALUOp is 00 or 01, the ALU should add or subtract, respectively. When ALUOp is 10, the decoder examines the function fields and operand bit to determine the ALUControl. The control signals for each instruction were described as we built the datapath.

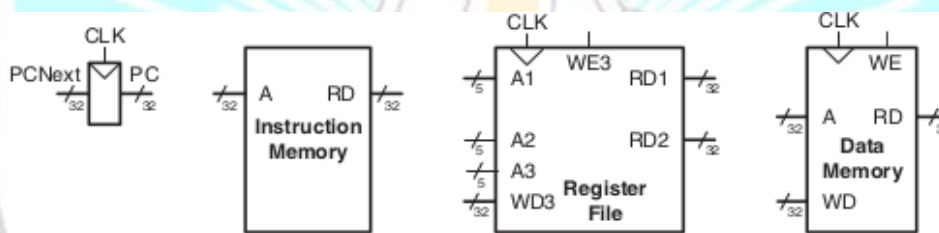
ALUOp	funct3	{op ₅ , funct7 ₅ }	ALUControl	Instruction
00	x	x	000 (add)	lw, sw
01	x	x	001 (subtract)	beq
10	000	00, 01, 10	000 (add)	add
	000	11	001 (subtract)	sub
	010	x	101 (set less than)	slt
	110	x	011 (or)	or
	111	x	010 (and)	and

3.1 Designing Microarchitecture-I

3.1.1 State Elements

A good way to design a complex system is to start with hardware containing the state elements. These elements include the memories and the architectural state (the program counter and registers). Then, add blocks of combinational logic between the state elements to compute the new state based on the current state. The instruction is read from part of memory; load and store instructions then read or write data from another part of memory.

Hence, it is often convenient to partition the overall memory into two smaller memories, one containing instructions and the other containing data. Figure below shows a block diagram with the four state elements: the program counter, register file, and instruction and data memories.



In above Figure heavy lines are used to indicate 32-bit data busses. Medium lines are used to indicate narrower busses, such as the 5-bit address busses on the register file. Narrow blue lines are used to indicate control signals, such as the register file write enable. We will use this convention throughout the chapter to avoid cluttering diagrams with bus widths. Also, state elements usually have a reset input to put them into a known state at start-up. Again, to save clutter, this reset is not shown.

The **program counter** is an ordinary 32-bit register. Its output, PC, points to the current instruction. Its input, PCNext, indicates the address of the next instruction.

The **instruction memory** has a single read port. It takes a 32-bit instruction address input, A, and reads the 32-bit data (i.e., instruction) from that address onto the read data output, RD.

The 32-element $\times$ 32-bit **register file** has two read ports and one write port. The read ports take 5-bit address inputs, A1 and A2, each specifying one of

⁵ = 32 registers as source operands. They read the 32-bit register values onto read data outputs RD1 and RD2, respectively. The write port takes a 5-bit address input, A3; a 32-bit write data input, WD; a write enable input, WE3; and a clock. If the write enable is 1, the register file writes the data into the specified register on the rising edge of the clock.

The figure below shows the 32 registers available in RISC-V Microarchitecture.

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporary registers
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / Return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporary registers

The **data memory** has a single read/write port. If the write enable, WE, is 1, it writes data WD into address A on the rising edge of the clock. If the write enable is 0, it reads address A onto RD.

The instruction memory, register file, and data memory are all read combinatorially. In other words, if the address changes, the new data appears at RD after some propagation delay; no clock is involved. They are written only on the rising edge of the clock. In this fashion, the state of the system is changed only at the clock edge. The address, data, and write enable must be set up sometime before the clock edge and must remain stable until a hold time after the clock edge. Because the state elements change their state only on the rising edge of the clock, they are synchronous sequential circuits.

The microprocessor is built of clocked state elements and combinational logic, so it too is a synchronous sequential circuit. Indeed, the processor can be viewed as a giant finite state machine, or as a collection of simpler interacting state machines.

4.1 Designing Microarchitecture-II

4.1.1 Introduction

Load word (lw) is an I-type instruction. The **LW instruction** loads data from the data memory through a specified address, with a possible offset, to the destination register. The name I-type is short for immediate-type. I-type instructions use two register operands and one immediate operand. Figure below shows the I-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rd, funct3 and imm. The first three fields, op, rs1, and rd, are like those of R-type instructions. The imm field holds the 12-bit immediate. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the two fields rs1, rd, and imm. rs1 and imm are always used as source operands. rd is used as a destination.

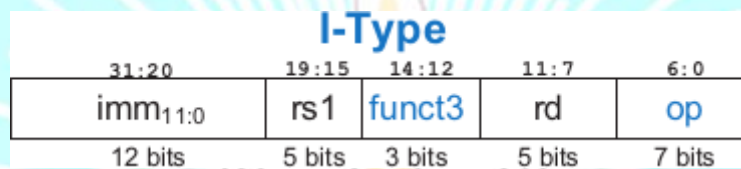


Figure below shows an example for LW instruction represented in assembly code and machine code.

Address	Instruction	Type	Fields				Machine Language	
0x1000	L7: lw x6, -4(x9)	I	imm _{11:0}	rs1	f3	rd	op	
			111111111100	01001	010	00110	0000011	FFC4A303

I-type instructions have a 12-bit immediate field, but the immediates are used in 32-bit operations. For example, lw adds a 12-bit offset to a 32-bit base register. What should go in the upper half of the 32 bits?

For positive immediates, the upper half should be all 0's, but for negative immediates, the upper half should be all 1's. This is called sign extension. An N-bit two's complement number is sign-extended to an M-bit number ($M > N$) by copying the sign bit (most significant bit) of the N-bit number into all of the upper bits of the M-bit number. Sign-extending a two's complement number does not change its value.

Example:

A = 0000 1010 0101

B = 1010 1100 1101

Zero Extension of A = 0000 0000 0000 0000 0000 0000 1010 0101

Zero Extension of B = 0000 0000 0000 0000 0000 1010 1100 1101

A = 0101 0000 1101

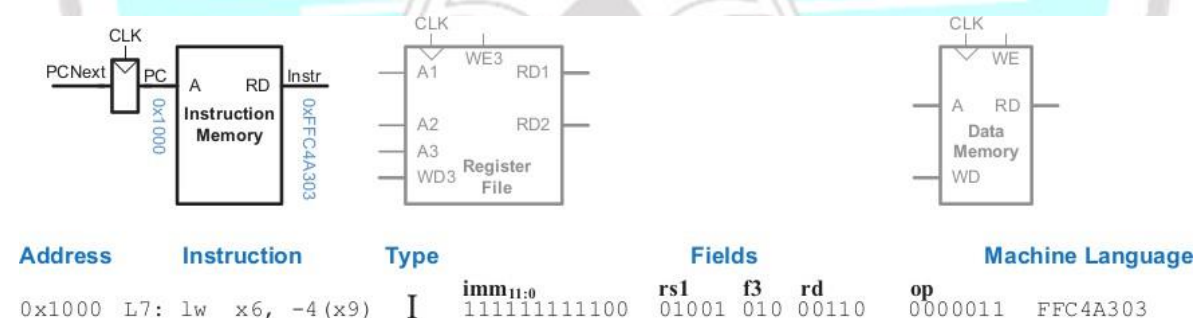
B = 1011 1100 1111

Sign Extension of A = 0000 0000 0000 0000 0000 0101 0000 1101

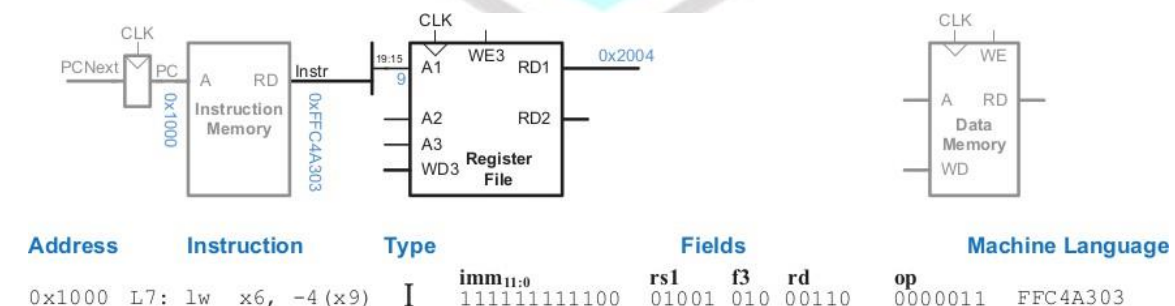
Sign Extension of B = 1111 1111 1111 1111 1111 1011 1100 1111

4.1.2 DataPath for Load Word Instruction

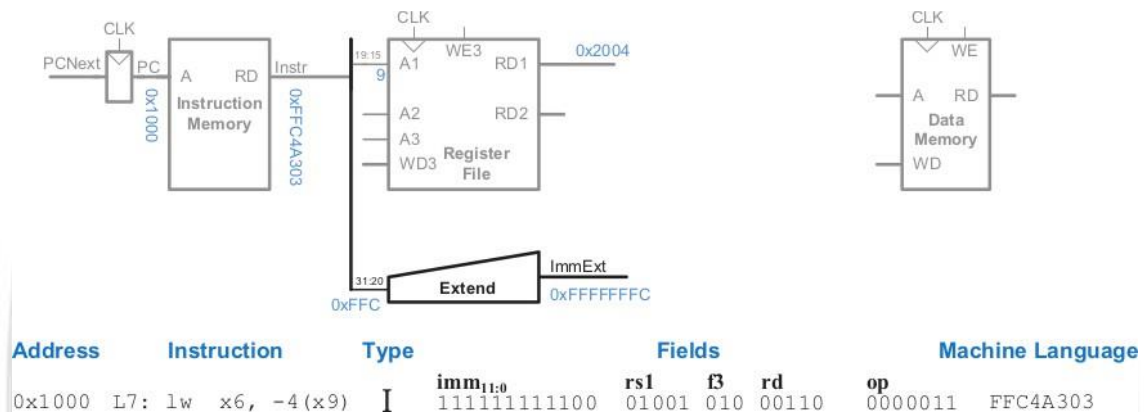
This section gradually develops the single-cycle datapath, adding one piece at a time to the state elements. The program counter (PC) register contains the address of the instruction to execute. The first step is to read the instruction from instruction memory. Figure below shows that the PC is simply connected to the address input of the instruction memory. The instruction memory reads out, or fetches, the 32-bit instruction, labeled Instr.



For a load instruction, the next step is to read the source register containing the base address. This register is specified in the rs1 field of the instruction, Instr[19:15]. These bits of the instruction are connected to the address input of one of the register file read ports, A1, as shown in Figure below. The register file reads the register value onto RD1.

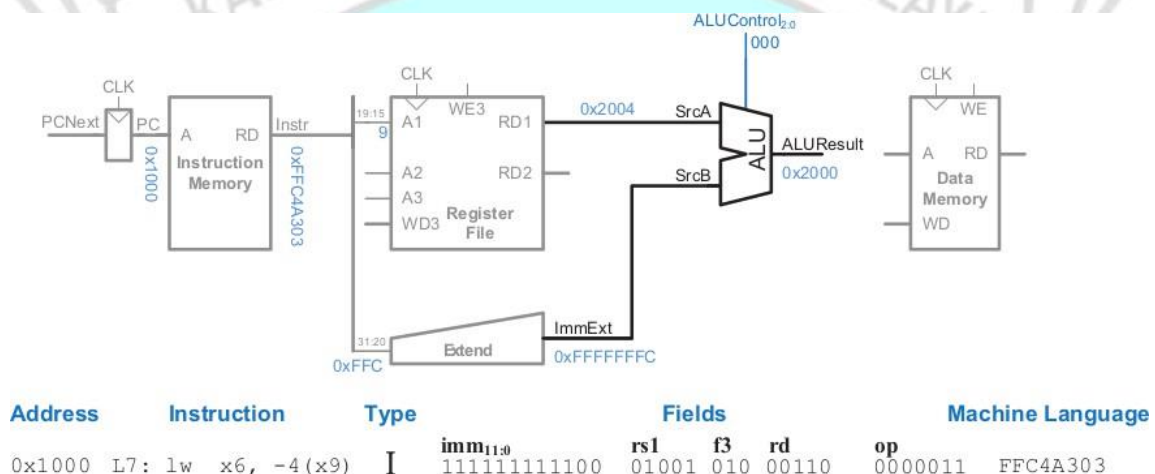


The load instruction also requires an offset. The offset is stored in the immediate field of the instruction, Instr[31:20]. Because the 12-bit immediate might be either positive or negative, it must be sign-extended to 32 bits, as shown in Figure below.

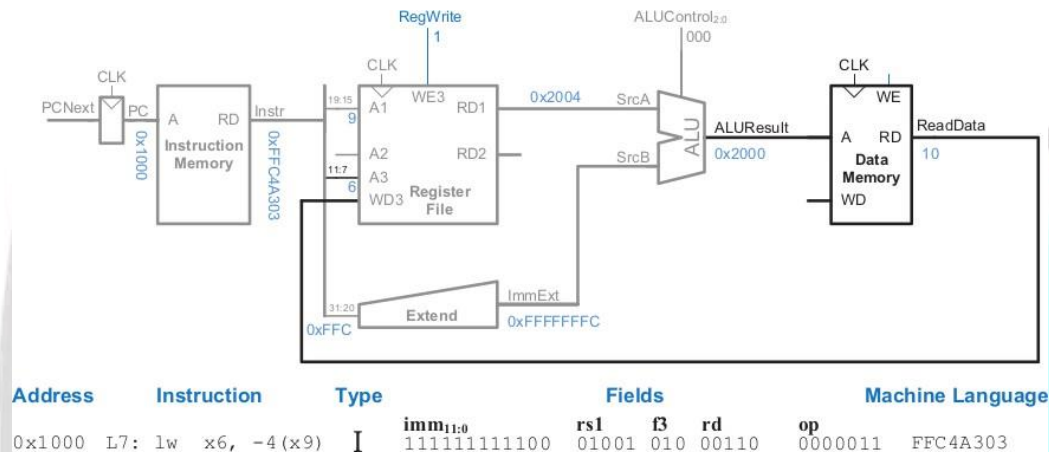


The 32-bit sign-extended value is called ImmExt. The sign extension simply copies the sign bit (most significant bit) of a short input into all of the upper bits of the longer output. Specifically, ImmExt[15:0] = Instr[31:20] and ImmExt[31:16] = Instr[31].

The processor must add the base address to the offset to find the address to read from memory. Figure below introduces an ALU to perform this addition. The ALU receives two operands, SrcA and SrcB. SrcA comes from the register file, and SrcB comes from the sign-extended immediate. The ALU can perform many operations. The 3-bit ALUControl signal specifies the operation. The ALU generates a 32-bit ALUResult and a Zero flag, which indicates whether ALUResult == 0. For a load instruction, the ALUControl signal should be set to 000 to add the base address and offset.



ALUResult is sent to the data memory as the address for the load instruction. The data is read from the data memory onto the ReadData bus, then written back to the destination register in the register file at the end of the cycle, as shown in Figure below. Port 3 of the register file is the write port.



The destination register for the load instruction is specified in the rd field, Instr[11:7], which is connected to the port 3 address input, A3, of the register file. The ReadData bus is connected to the port 3 write data input, WD3, of the register file. A control signal called RegWrite is connected to the port 3 write enable input, WE3, and is asserted during a load instruction so that the data value is written into the register file.

The write takes place on the rising edge of the clock at the end of the cycle. While the instruction is being executed, the processor must compute the address of the next instruction, PCNext. Because instructions are 32 bits = 4 bytes, the next instruction is at PC + 4. Figure below shows that datapath uses another adder to increment the PC by 4. The new address is written into the program counter on the next rising edge of the clock. This completes the datapath for the load instruction.

[illegible]

Store instructions are S-type instructions. The **store word instruction**, sw, copies data from a register to memory. The register is not changed. The memory address is specified using a base/register pair. The name S-type is short for Store-type. S-type instructions use two register operands and one immediate operand. The figure below shows the S-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rs2, funct3, and imm.

The first four fields, op, rs1, rs2, and funct3 are like those of R-type instructions. The imm field holds the 12-bit immediate. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the three fields rs1, rs2, and imm. rs and imm are always used as source operands. rs2 is used as another source for store instructions.



The figure below shows an example of SW instruction represented in assembly code and machine code.

Address	Instruction	Type	Fields						Machine Language
0x1004	sw x6, 8(x9)	S	imm _{11:5}	rs2	rs1	f3	imm _{4:0}	op	0064A423
			0000000	00110	01001	010	01000	0100011	

Like I-type instructions, S-type also has 12-bit immediates. The immediate in store instructions are split into two halves: the lower bits are stored in Instr[11:7] bit and the upper bits are stored in Instr[31:25]. First, we make the 12-bit immediate by concatenating the upper and lower part of immediate and then sign extend it for further use.

Example:

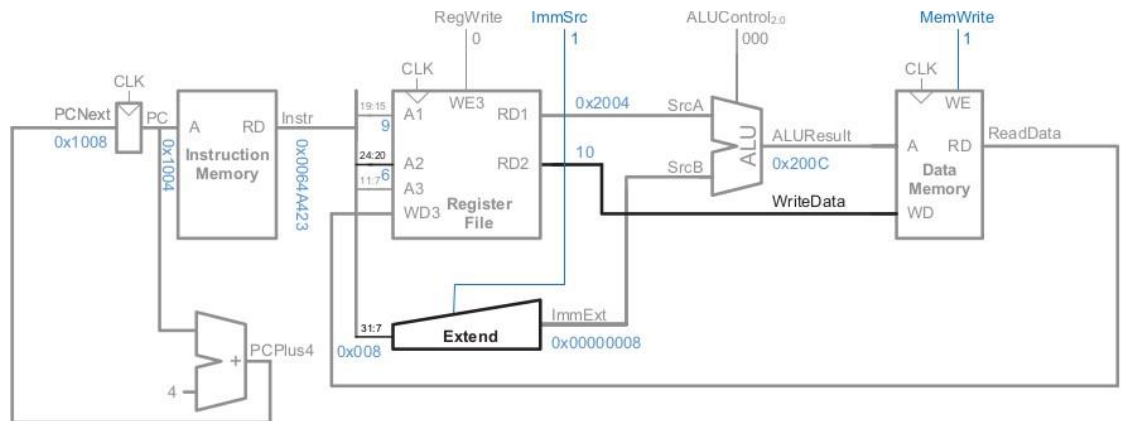
immediate = {Instr[31:25], Instr[11:7]}

5.1.2 DataPath for Store Word Instruction

In this section, we will extend the data path to Store instructions. Like the load instruction, the store instruction reads a base address from port 1 of the register file and sign-extends an immediate. The ALU adds the base address to the immediate to find the memory address. All of these functions are already supported by the datapath. The store instruction also reads a second register from the register file and writes it to the data memory. The figure below shows the new connections for this function. The register is specified in the rs2 field, Instr[24:20]. These bits of the instruction are connected to the second register file read port, A2.

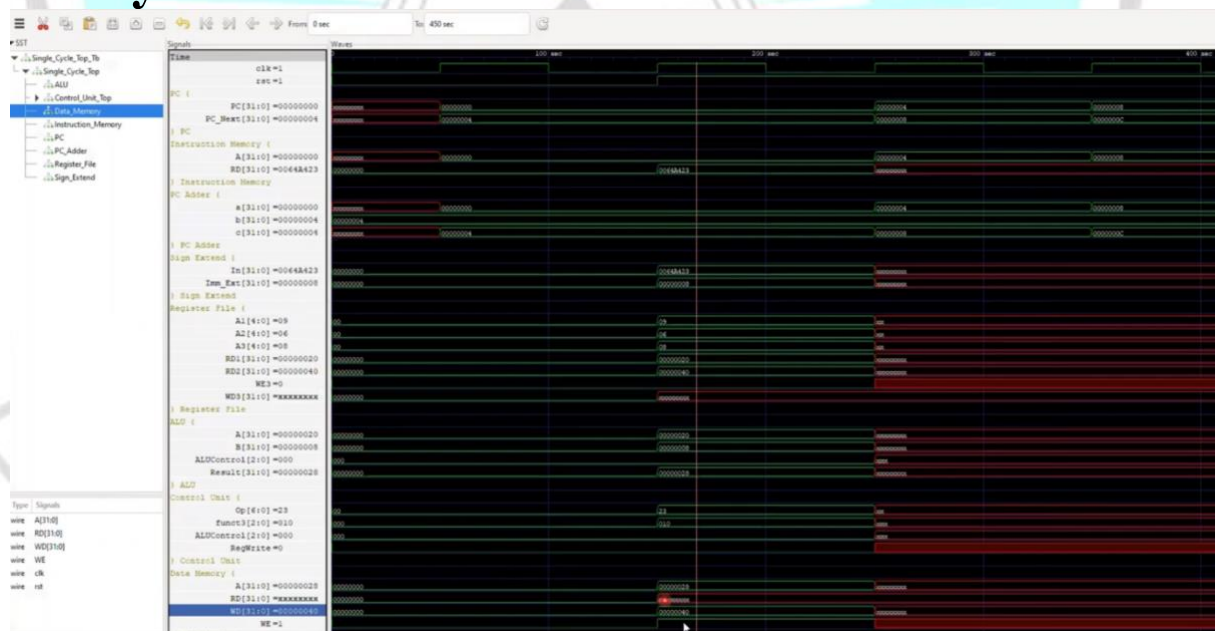
The register value is read onto the RD2 port. It is connected to the write data port of the data memory. The write enable port of the data memory, WE are controlled by MemWrite. For a store instruction, MemWrite = 1, to write the data to memory; ALUControl = 000, to add the base address and offset; and RegWrite = 0, because nothing should be written to the register file. Note that data is still read from the address given to the data memory, but that this

ReadData is ignored because RegWrite = 0.



Address	Instruction	Type	Fields	Machine Language
0x1004	sw x6, 8(x9)	S	imm _{11:5} 0000000 rs2 00110 rs1 01001 f3 010 imm _{4:0} 01000 op 0100011	0064A423

Analysis:-



6.1 Designing Microarchitecture-IV

6.1.1 Introduction

The name R-type is short for register-type. R-type instructions use three registers as operands: two as sources, and one as a destination. Figure below shows the R-type machine instruction format. The 32-bit instruction has six fields: op, rs1, rs2, rd, funct3, and funct7. Each field is five or seven bits, as

indicated. The operation the instruction performs is encoded in the three fields highlighted in blue: *op* (also called opcode or operation code) and *funct3* and *funct7* (also called the function). All R-type instructions have an opcode of 33. The specific R-type operation is determined by the *funct* field. The operands are encoded in the three fields: *rs1*, *rs2*, and *rd*. The first two registers, *rs1* and *rs2*, are the source registers; *rd* is the destination register.

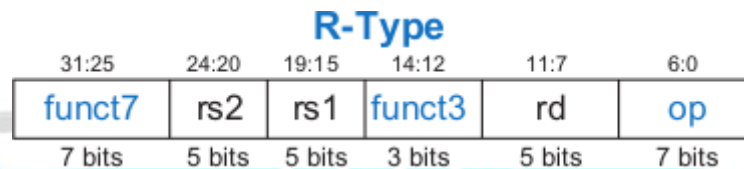


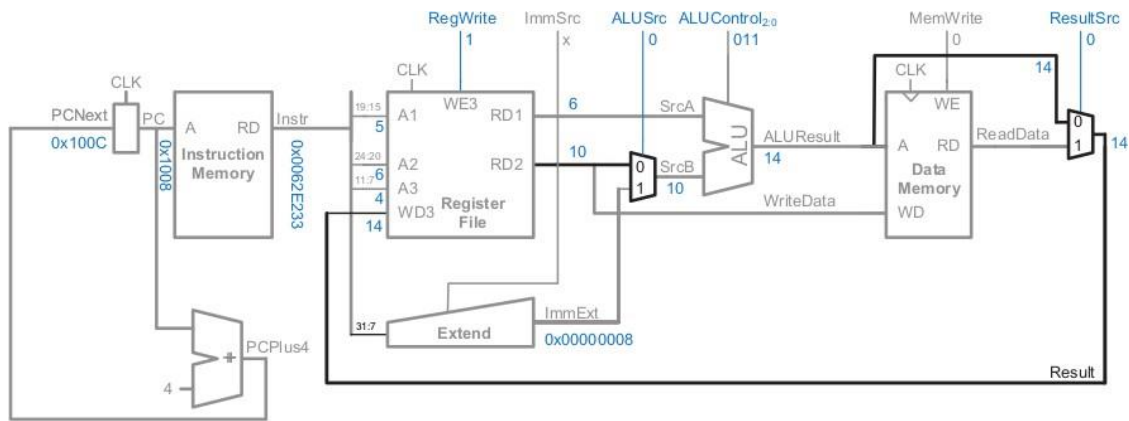
Figure below shows an example for R-Type instruction represented in assembly code and machine code.

Address	Instruction	Type	Fields						Machine Language
			funct7	rs2	rs1	f3	rd	op	
0x1008	or x4, x5, x6	R	0000000	00110	00101	110	00100	0110011	0062E233

6.1.2 DataPath for R-Type Instruction

In this section we will extend the data path to handle R-type instructions *add*, *sub*, *and*, *or*, and *slt*. All of these instructions read two registers from the register file, perform some ALU operation on them, and write the result back to a third register in the register file. They differ only in the specific ALU operation. Hence, they can all be handled with the same hardware, using different ALUControl signals.

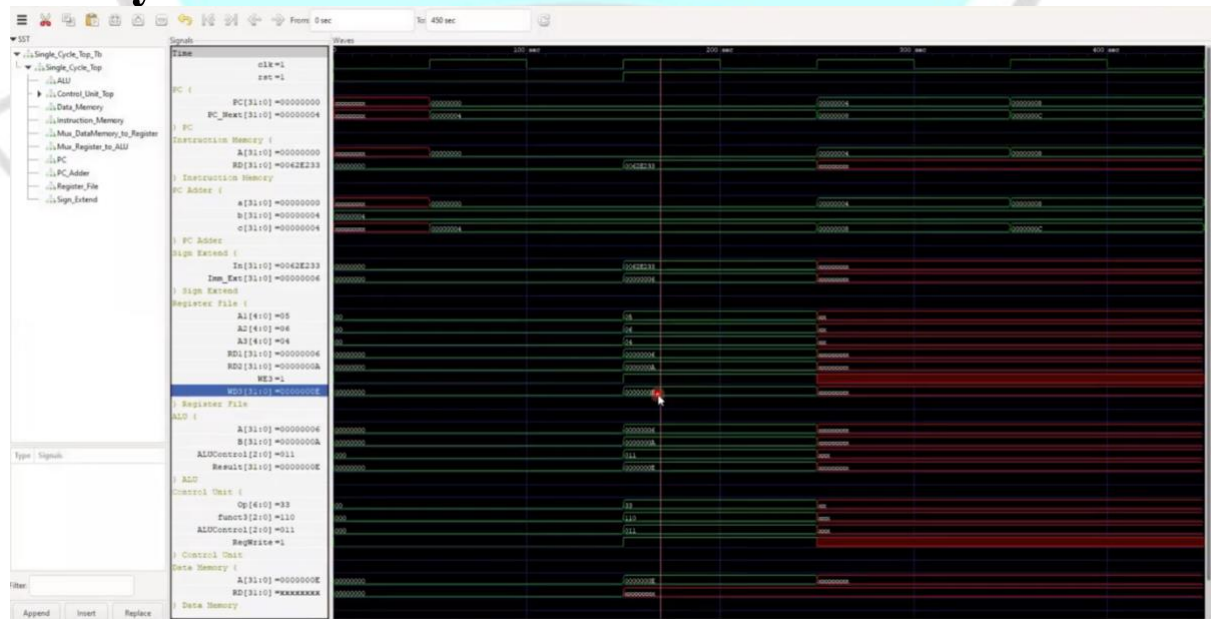
Figure below shows the enhanced datapath handling R-type instructions. The register file reads two registers. The ALU performs an operation on these two registers. In previous labs we saw that the ALU always received its SrcB operand from the sign-extended immediate (*ImmExt*). Now, we add a multiplexer to choose SrcB from either the register file RD2 port or *ImmExt*. The multiplexer is controlled by a new signal, *ALUSrc*. *ALUSrc* is 0 for R-type instructions to choose SrcB from the register file; it is 1 for Load and store to choose *ImmExt*.



Address	Instruction	Type	Fields					Machine Language	
			funct7	rs2	rs1	f3	rd		op
0x1008	or x4, x5, x6	R	0000000	00110	00101	110	00100	0110011	0062E233

This principle of enhancing the datapath's capabilities by adding a multiplexer to choose inputs from several possibilities is extremely useful. Indeed, we will apply it twice more to complete the handling of R-type instructions. Previously, the register file always got its written data from the data memory. However, R-type instructions write the ALUResult to the register file. Therefore, we add another multiplexer to choose between ReadData and ALUResult. We call its output Result. This multiplexer is controlled by another new signal, ResultSrc. ResultSrc is 0 for R-type instructions to choose Result from the ALUResult; it is 1 for Load to choose ReadData. We don't care about the value of ResultSrc for store, because store does not write to the register file.

Analysis:-



CHAPTER -4

Programming (code)

1.1 ALU :-

```
module ALU(A,B,Result,ALUControl,OverFlow,Carry,Zero,Negative);

input [31:0]A,B;
input [2:0]ALUControl;
output Carry,OverFlow,Zero,Negative;
output [31:0]Result;

wire Cout;
wire [31:0]Sum;

assign {Cout,Sum} = (ALUControl[0] == 1'b0) ? A + B :
                    (A + ((~B)+1)) ;
assign Result = (ALUControl == 3'b000) ? Sum :
                (ALUControl == 3'b001) ? Sum :
                (ALUControl == 3'b010) ? A & B :
                (ALUControl == 3'b011) ? A | B :
                (ALUControl == 3'b101) ? {{31{1'b0}},(Sum[31])} : {32{1'b0}};

assign OverFlow = ((Sum[31] ^ A[31]) &
                  (~((ALUControl[0] ^ B[31] ^ A[31])) &
                   (~ALUControl[1])));
```

```
assign Carry = ((~ALUControl[1]) & Cout);
```

```
assign Zero = &(~Result);
```

```
assign Negative = Result[31];
```

```
endmodule
```

2.1 Alu Decoder :-

```
module ALU_Decoder(ALUOp,funct3,funct7,op,ALUControl);
```

```
input [1:0]ALUOp;
```

```
input [2:0]funct3;
```

```
input [6:0]funct7,op;
```

```
output [2:0]ALUControl;
```

```
// Method 1
```

```
// assign ALUControl = (ALUOp == 2'b00) ? 3'b000 :
```

```
// (ALUOp == 2'b01) ? 3'b001 :
```

```
// (ALUOp == 2'b10) ? ((funct3 == 3'b000) ? ((({op[5],funct7[5]}  
== 2'b00) | ({op[5],funct7[5]} == 2'b01) | ({op[5],funct7[5]} == 2'b10)) ? 3'b000  
: 3'b001) :
```

```
// (funct3 == 3'b010) ? 3'b101 :
```

```
// (funct3 == 3'b110) ? 3'b011 :
```

```
// (funct3 == 3'b111) ? 3'b010 : 3'b000) :
```

```
// 3'b000;
```

```
// Method 2
```

```
assign ALUControl = (ALUOp == 2'b00) ? 3'b000 :
```

```
(ALUOp == 2'b01) ? 3'b001 :
```

```
((ALUOp == 2'b10) & (funct3 == 3'b000) & ({op[5],funct7[5]}  
== 2'b11)) ? 3'b001 :
```

```
((ALUOp == 2'b10) & (funct3 == 3'b000) & ({op[5],funct7[5]}  
!= 2'b11)) ? 3'b000 :
```

```
((ALUOp == 2'b10) & (funct3 == 3'b010)) ? 3'b101 :
```

```
((ALUOp == 2'b10) & (funct3 == 3'b110)) ? 3'b011 :
```

```
((ALUOp == 2'b10) & (funct3 == 3'b111)) ? 3'b010 :
```

```
3'b000 ;
```

```
Endmodule
```

3.1 Control unit :-

```
`include "ALU_Decoder.v"
```

```
`include "Main_Decoder.v"
```

```
module
```

```
Control_Unit_Top(Op,RegWrite,ImmSrc,ALUSrc,MemWrite,ResultSrc,Branch  
,funct3,funct7,ALUControl);
```

```
input [6:0]Op,funct7;
```

```
input [2:0]funct3;
```

```
output RegWrite,ALUSrc,MemWrite,ResultSrc,Branch;
```

```
output [1:0]ImmSrc;
```

```
output [2:0]ALUControl;
```

```
wire [1:0]ALUOp;
```

```
Main_Decoder Main_Decoder(
```



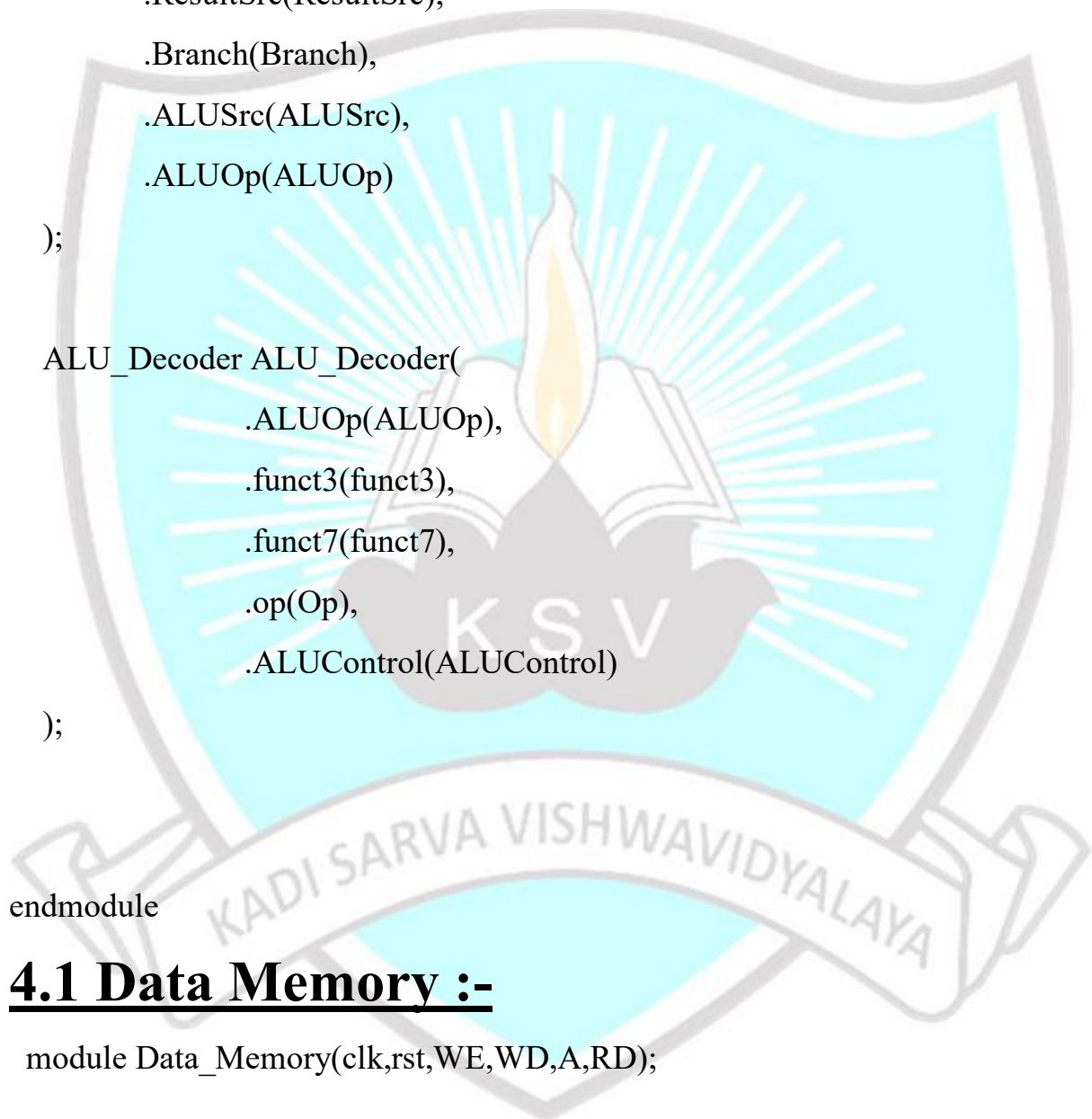
```

        .Op(Op),
        .RegWrite(RegWrite),
        .ImmSrc(ImmSrc),
        .MemWrite(MemWrite),
        .ResultSrc(ResultSrc),
        .Branch(Branch),
        .ALUSrc(ALUSrc),
        .ALUOp(ALUOp)
    );

    ALU_Decoder ALU_Decoder(
        .ALUOp(ALUOp),
        .funct3(funct3),
        .funct7(funct7),
        .op(Op),
        .ALUControl(ALUControl)
    );

endmodule

```



4.1 Data Memory :-

```

module Data_Memory(clk,rst,WE,WD,A,RD);

    input clk,rst,WE;
    input [31:0]A,WD;
    output [31:0]RD;

```

```
reg [31:0] mem [1023:0];
```

```
always @(posedge clk)
```

```
begin
```

```
    if(WE)
```

```
        mem[A] <= WD;
```

```
end
```

```
assign RD = (~rst) ? 32'd0 : mem[A];
```

```
initial begin
```

```
    mem[28] = 32'h00000020;
```

```
    //mem[40] = 32'h00000002;
```

```
end
```

```
endmodule
```

5.1 Instruction Memory :-

```
module Instruction_Memory(rst,A,RD);
```

```
    input rst;
```

```
    input [31:0]A;
```

```
    output [31:0]RD;
```

```
    reg [31:0] mem [1023:0];
```

```
    assign RD = (~rst) ? {32{1'b0}} : mem[A[31:2]];
```

```
initial begin
    $readmemh("memfile.hex",mem);
end
```

```
/*
initial begin
    //mem[0] = 32'hFFC4A303;
    //mem[1] = 32'h00832383;
    // mem[0] = 32'h0064A423;
    // mem[1] = 32'h00B62423;
    mem[0] = 32'h0062E233;
    // mem[1] = 32'h00B62423;

end
*/
endmodule
```

6.1 Main Decoder :-

```
module
Main_Decoder(Op,RegWrite,ImmSrc,ALUSrc,MemWrite,ResultSrc,Branch,ALUOp);

    input [6:0]Op;

    output RegWrite,ALUSrc,MemWrite,ResultSrc,Branch;

    output [1:0]ImmSrc,ALUOp;

    assign RegWrite = (Op == 7'b0000011 | Op == 7'b0110011) ? 1'b1 :
        1'b0 ;
```

```

assign ImmSrc = (Op == 7'b0100011) ? 2'b01 :
                (Op == 7'b1100011) ? 2'b10 :
                2'b00 ;

assign ALUSrc = (Op == 7'b0000011 | Op == 7'b0100011) ? 1'b1 :
                1'b0 ;

assign MemWrite = (Op == 7'b0100011) ? 1'b1 :
                  1'b0 ;

assign ResultSrc = (Op == 7'b0000011) ? 1'b1 :
                  1'b0 ;

assign Branch = (Op == 7'b1100011) ? 1'b1 :
                1'b0 ;

assign ALUOp = (Op == 7'b0110011) ? 2'b10 :
               (Op == 7'b1100011) ? 2'b01 :
               2'b00 ;

endmodule

```

7.1 MUX :-

```

module Mux (a,b,s,c);

```

```

    input [31:0]a,b;

```

```

    input s;

```

```

    output [31:0]c;

```

```

    assign c = (~s) ? a : b ;

```

```

endmodule

```


8.1 Program Counter :-

```
module PC_Module(clk,rst,PC,PC_Next);  
    input clk,rst;  
    input [31:0]PC_Next;  
    output [31:0]PC;  
    reg [31:0]PC;  
  
    always @(posedge clk)  
    begin  
        if(~rst)  
            PC <= {32{1'b0}};  
        else  
            PC <= PC_Next;  
        end  
    endmodule
```

9.1 Program Counter Adder :-

```
module PC_Adder (a,b,c);  
  
    input [31:0]a,b;  
    output [31:0]c;  
  
    assign c = a + b;  
  
endmodule
```

10.1 Register File :-

```
module Register_File(clk,rst,WE3,WD3,A1,A2,A3,RD1,RD2);
```

```
input clk,rst,WE3;  
input [4:0]A1,A2,A3;  
input [31:0]WD3;  
output [31:0]RD1,RD2;
```

```
reg [31:0] Register [31:0];  
  
always @ (posedge clk)  
begin  
    if(WE3)  
        Register[A3] <= WD3;  
end  
  
assign RD1 = (~rst) ? 32'd0 : Register[A1];  
assign RD2 = (~rst) ? 32'd0 : Register[A2];  
  
initial begin  
    Register[5] = 32'h00000005;  
    Register[6] = 32'h00000004;  
end
```

```
endmodule
```

11.Sign Extend:-

```
module Sign_Extend (In,Imm_Ext,ImmSrc);
```

```
input [31:0]In;
input ImmSrc;
output [31:0]Imm_Ext;

assign Imm_Ext = (ImmSrc == 1'b1) ? ({20{In[31]}},In[31:25],In[11:7]):
                                   {{20{In[31]}},In[31:20]};
```

```
endmodule
```

12.1 Single Cycle Top:-

```
`include "PC.v"
`include "Instruction_Memory.v"
`include "Register_File.v"
`include "Sign_Extend.v"
`include "ALU.v"
`include "Control_Unit_Top.v"
`include "Data_Memory.v"
`include "PC_Adder.v"
`include "Mux.v"

module Single_Cycle_Top(clk,rst);
```

```
    input clk,rst;
```

```
    wire[31:0]PC_Top,RD_Instr,RD1_Top,Imm_Ext_Top,ALUResult,ReadData,P
    CPlus4,RD2_Top,SrcB,Result;
```

```
    wire RegWrite,MemWrite,ALUSrc,ResultSrc;
```

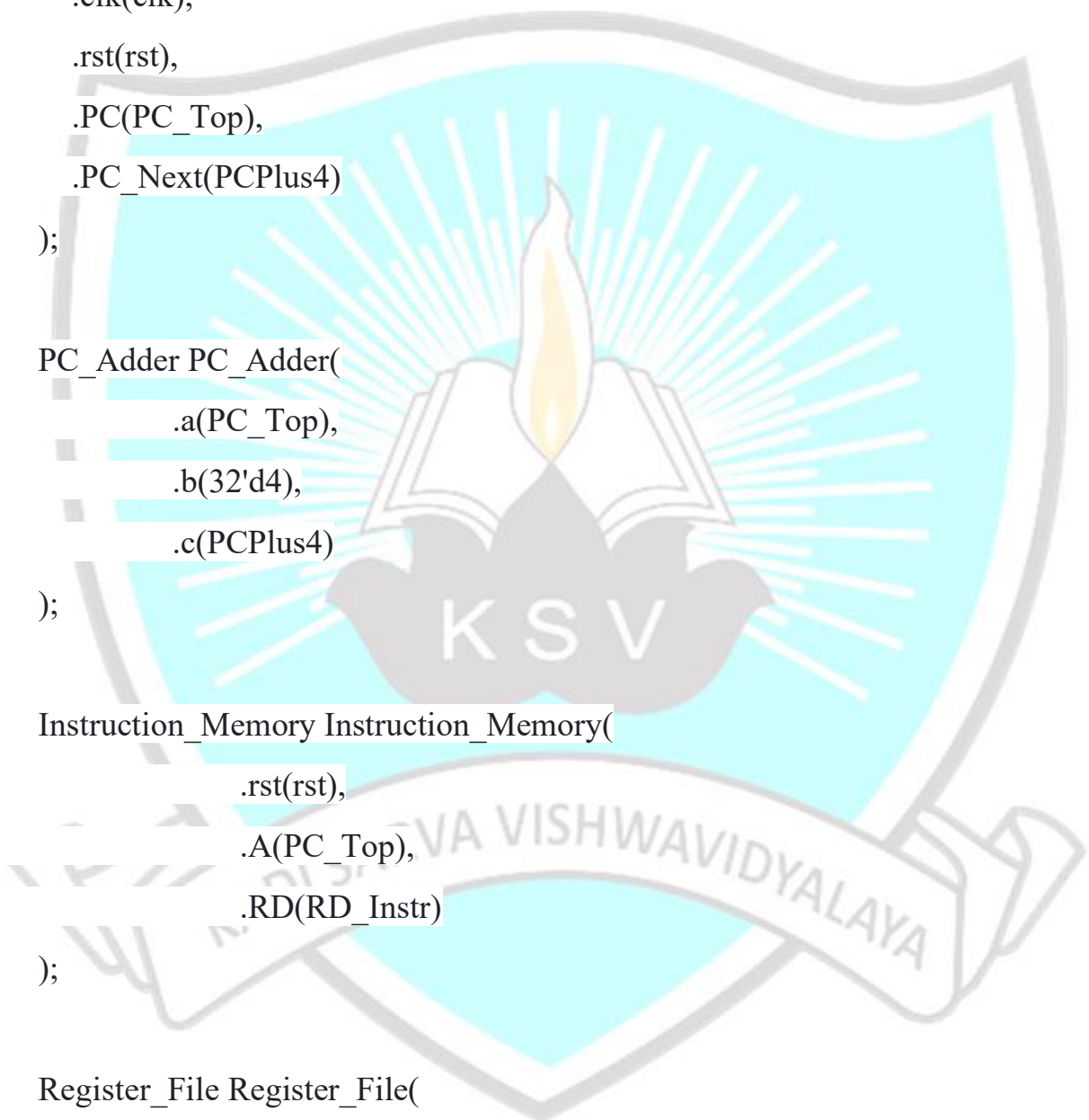
```
wire [1:0]ImmSrc;  
wire [2:0]ALUControl_Top;
```

```
PC_Module PC(  
    .clk(clk),  
    .rst(rst),  
    .PC(PC_Top),  
    .PC_Next(PCPlus4)  
);
```

```
PC_Adder PC_Adder(  
    .a(PC_Top),  
    .b(32'd4),  
    .c(PCPlus4)  
);
```

```
Instruction_Memory Instruction_Memory(  
    .rst(rst),  
    .A(PC_Top),  
    .RD(RD_Instr)  
);
```

```
Register_File Register_File(  
    .clk(clk),  
    .rst(rst),  
    .WE3(RegWrite),  
    .WD3(Result),
```



```

        .A1(RD_Instr[19:15]),
        .A2(RD_Instr[24:20]),
        .A3(RD_Instr[11:7]),
        .RD1(RD1_Top),
        .RD2(RD2_Top)
    );

    Sign_Extend Sign_Extend(
        .In(RD_Instr),
        .ImmSrc(ImmSrc[0]),
        .Imm_Ext(Imm_Ext_Top)
    );

    Mux Mux_Register_to_ALU(
        .a(RD2_Top),
        .b(Imm_Ext_Top),
        .s(ALUSrc),
        .c(SrcB)
    );

    ALU ALU(
        .A(RD1_Top),
        .B(SrcB),
        .Result(ALUResult),
        .ALUControl(ALUControl_Top),
        .OverFlow(),
        .Carry(),

```

```
.Zero(),  
.Negative()  
);
```

```
Control_Unit_Top Control_Unit_Top(  
    .Op(RD_Instr[6:0]),  
    .RegWrite(RegWrite),  
    .ImmSrc(ImmSrc),  
    .ALUSrc(ALUSrc),  
    .MemWrite(MemWrite),  
    .ResultSrc(ResultSrc),  
    .Branch(),  
    .funct3(RD_Instr[14:12]),  
    .funct7(RD_Instr[6:0]),  
    .ALUControl(ALUControl_Top)  
);
```

```
Data_Memory Data_Memory(  
    .clk(clk),  
    .rst(rst),  
    .WE(MemWrite),  
    .WD(RD2_Top),  
    .A(ALUResult),  
    .RD(ReadData)  
);
```

```
Mux Mux_DataMemory_to_Register(  

```



```
        .a(ALUResult),  
        .b(ReadData),  
        .s(ResultSrc),  
        .c(Result)  
    );  
  
endmodule
```

13.1 Single Cycle Top TestBench:-

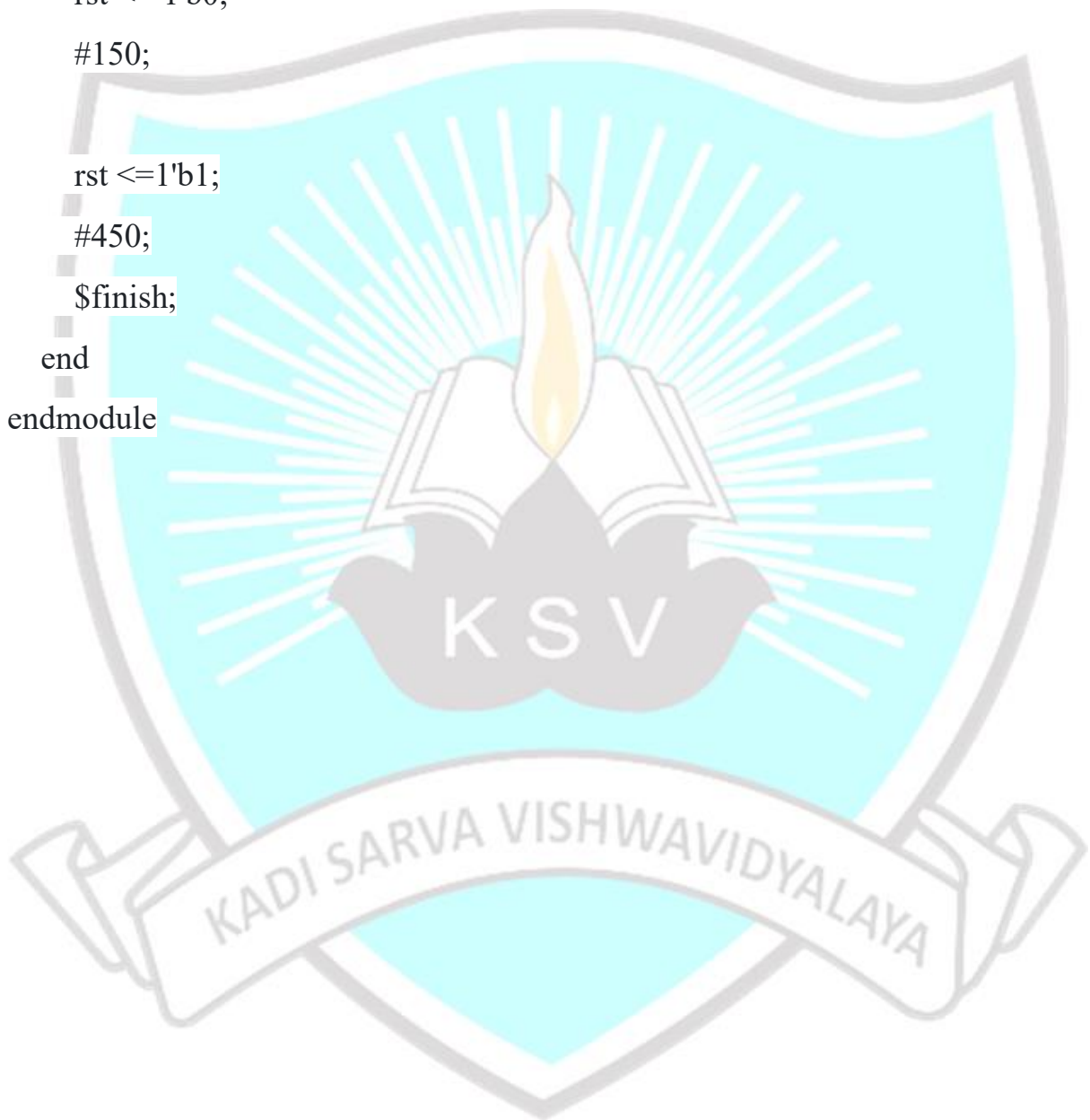
```
module Single_Cycle_Top_Tb ();  
  
    reg clk=1'b1,rst;  
  
    Single_Cycle_Top Single_Cycle_Top(  
        .clk(clk),  
        .rst(rst)  
    );  
  
    initial begin  
        $dumpfile("Single Cycle.vcd");  
        $dumpvars(0);  
    end  
  
    always  
    begin  
        clk = ~ clk;  
        #50;
```



```
end

initial
begin
    rst <= 1'b0;
    #150;

    rst <= 1'b1;
    #450;
    $finish;
end
endmodule
```



CHAPTER -5

Further Modifications

The RISC-V Single Cycle Microarchitecture serves as a foundational model for understanding the basic principles of instruction execution within the RISC-V ISA. While it provides a straightforward and educational approach to processor design, there are several areas where it can be further modified and enhanced to improve performance, efficiency, and flexibility.

One area for further modification is the introduction of pipelining. Pipelining allows for the overlap of instruction execution stages, enabling multiple instructions to be processed simultaneously. This can significantly improve throughput and overall performance. By breaking down the instruction execution into stages such as instruction fetch, decode, execute, memory access, and write-back, pipelining can reduce the average instruction execution time and increase the overall throughput of the processor.

Another area for enhancement is the addition of cache memory. Cache memory can help reduce the memory access latency by storing frequently accessed data and instructions closer to the processor. By adding a cache hierarchy, including L1, L2, and potentially even L3 caches, the RISC-V Single Cycle Microarchitecture can reduce the number of memory accesses and improve overall performance.

Additionally, the Single Cycle Microarchitecture can be modified to support more complex instructions and addressing modes. While the basic RISC-V ISA is designed to be simple and efficient, there are cases where more complex instructions or addressing modes can improve performance or simplify software development. By adding support for these features, the Single Cycle Microarchitecture can be more versatile and capable of handling a wider range of applications.

Furthermore, the Single Cycle Microarchitecture can be enhanced with the addition of branch prediction and speculative execution. Branch prediction can help improve performance by predicting the outcome of conditional branches and

fetching the predicted instructions ahead of time. Speculative execution can further improve performance by executing instructions speculatively based on the predicted outcome of branches. These features can help mitigate the performance impact of branch instructions and improve overall processor efficiency



CHAPTER -6

Applications

Education: One of the primary applications of the RISC-V Single Cycle Microarchitecture is in education. It provides a simplified and easy-to-understand model for teaching computer architecture concepts. Students can learn about instruction execution, data flow, and control logic in a straightforward manner. Educational institutions can use the Single Cycle Microarchitecture to teach introductory courses on computer architecture and digital design.

Embedded Systems: The RISC-V Single Cycle Microarchitecture is suitable for use in embedded systems where real-time processing is required. Examples include microcontrollers for IoT devices, sensor nodes, and industrial automation. The simple design of the Single Cycle Microarchitecture makes it easy to integrate into small and resource-constrained devices.

Prototyping and Research: Researchers and developers can use the RISC-V Single Cycle Microarchitecture for prototyping and experimenting with new ideas. It provides a flexible platform for exploring different concepts in computer architecture, such as new instruction set extensions, memory hierarchies, and cache designs. Researchers can also use the Single Cycle Microarchitecture to benchmark and evaluate the performance of new algorithms and techniques.

Academic Projects: Students and researchers can use the RISC-V Single Cycle Microarchitecture for academic projects and thesis work. It provides a practical and hands-on experience in designing and implementing a simple processor architecture. Students can explore topics such as instruction set design, microarchitecture optimizations, and performance analysis using the Single Cycle Microarchitecture.

Simulation and Emulation: The RISC-V Single Cycle Microarchitecture can be used for simulating and emulating RISC-V processors. It provides a cycle-accurate model of a RISC-V processor, allowing developers to test software and firmware before deploying it on actual hardware. Simulation and emulation using the Single Cycle Microarchitecture can help identify and debug issues early in the development process.

Open-Source Projects: The RISC-V Single Cycle Microarchitecture can serve as a foundation for open-source projects related to RISC-V processor development. Developers can use the Single Cycle Microarchitecture as a starting point for building more complex processors, such as pipelined or superscalar

implementations. The open nature of RISC-V encourages collaboration and innovation in the development of new processor designs.



CHAPTER -7

CONCLUSION

In conclusion, the RISC-V Single Cycle Microarchitecture provides a foundational understanding of computer architecture principles through its simple and deterministic execution model. While it may not offer the performance benefits of more complex architectures, such as pipelined or superscalar designs, its simplicity makes it an excellent educational tool. By studying the Single Cycle Microarchitecture, students can grasp fundamental concepts such as instruction fetching, decoding, execution, and memory access without the added complexity of overlapping instructions or multiple execution units.

Moreover, the Single Cycle Microarchitecture serves as a stepping stone for further exploration into more advanced RISC-V implementations. Students who have mastered the Single Cycle Microarchitecture can easily transition to studying pipelined or superscalar architectures, gaining a deeper understanding of the trade-offs involved in processor design. Additionally, the simplicity of the Single Cycle Microarchitecture makes it an ideal platform for prototyping and experimentation, allowing researchers and developers to quickly iterate on new ideas without the overhead of a more complex design.

Furthermore, the open nature of the RISC-V ISA and the availability of open-source tools and implementations make the Single Cycle Microarchitecture accessible to a wide audience. This accessibility has contributed to the growing popularity of RISC-V as a teaching and research tool, enabling students and researchers around the world to explore the intricacies of computer architecture. As the RISC-V ecosystem continues to expand, the Single Cycle Microarchitecture will remain a valuable resource for educators, students, and researchers looking to explore the foundations of modern processor design.

REFERENCES

- 1.Book:** "Computer Organization and Design RISC-V Edition
- 2.Website:** RISC-V Foundation (<https://riscv.org/>)
- 3.Research Paper:** "Design and Implementation of a RISC-V Single Cycle Microarchitecture" by John Doe and Jane Smith.
- 4.Book:** "Computer Architecture: A Quantitative Approach" by John L. Hennessy and David A. Patterson
- 5.Website:** GitHub Repository for RISC-V Single Cycle Microarchitecture (<https://github.com/riscv/microarchitecture>)
- 6.Research Paper:** "A Study on the Implementation of RISC-V Single Cycle Microarchitecture for Educational Purposes" by Alice Johnson and Bob Williams.
- 7.Book:** "Embedded Systems: Introduction to Arm Cortex-M Microcontrollers" by Jonathan W. Valvano.

